Project Specification Document

Zero-Blast Precision Volume Controller — AI Implementation Guide

This document contains the complete technical architecture, Cursor AI personas, unprompted edge cases, and code templates required to build a master-level Chrome Extension for granular audio manipulation.

1. AI Personas (For Cursor .cursorrules)

Provide these personas to the Cursor agent to establish the correct technical context and avoid boilerplate code traps.

> **Persona A: The Extension Architect**
> "You are an expert in Chrome Extension Manifest V3. You prioritize non-persistent background service workers, clean `chrome.storage.local` state management, and robust message passing between the popup and content scripts. You avoid `tabCapture` to prevent native muting, opting instead to inject logic directly into the DOM."

> **Persona B: The DSP (Digital Signal Processing) Engineer**
> "You are a Web Audio API specialist. You understand GainNodes, cross-origin resource sharing (CORS) as it relates to `MediaElementSource`, phase inversion (negative gain values), and logarithmic fader curves. You prioritize audio safety, ensuring no 'clipping' occurs during volume transitions."

2. Core Functional Requirements

- **Zero-Blast Initialization:** The moment the extension hooks into a media element, it must immediately seize the AudioContext and set `gain.value = 0`. Audio must be manually ramped up by the user.
- **Hybrid Precision Input:** The UI must include a logarithmic slider (fader curve) where 0% to 20% occupies 50% of the physical slider width. It must also include a numeric text input that accepts floating-point numbers (e.g., 5.5%).
- **Shift-Modifier for Micro-adjustments:** Holding the `Shift` key while dragging the slider reduces the increment scale by 10x for granular background volume tuning.
- **Phase Inversion Support:** The numeric input must allow negative numbers. If a negative number is entered, the UI should indicate "Phase Inverted" rather than failing validation.

- **Global Keyboard Shortcuts:** Implement `chrome.commands` for 'Increase Volume', 'Decrease Volume', and 'Panic/Mute'.

3. The "Unasked For" Features (Crucial Edge Cases)

To ensure this operates like professional software, the following must be implemented by the AI:

- **The "One Source" Protection:** A `<video>` element can only be connected to an AudioContext once. The AI must implement a check (e.g., `if (window.audioNodeInjected) return;`) to prevent crash loops if the user opens the popup multiple times.
- **Visual On-Screen Display (OSD):** When using keyboard shortcuts, the popup isn't visible. The content script must inject a temporary, non-blocking floating `div` in the top right of the active tab showing the current percentage (fading out after 2 seconds).
- **Dynamic DOM Mutation Observer:** Single Page Applications (like YouTube or Spotify) destroy and recreate video tags when navigating between songs/videos without reloading the page. The extension must use a `MutationObserver` to automatically hook into new media elements as they appear.

4. Critical Code Snippets

A. The Logarithmic Fader Curve (JavaScript)

This snippet maps a standard 0-100 range slider to an exponential curve, giving massive precision at low volumes.

```
// Convert linear slider position (0-100) to logarithmic gain (0-1)
function sliderToGain(sliderValue) {
    // x^2 / 10000 gives a smooth curve prioritizing the low end
    let normalized = sliderValue / 100;
    return Math.pow(normalized, 2);
}

// Convert target gain (0-1) back to slider position (0-100)
function gainToSlider(gainValue) {
    return Math.sqrt(gainValue) * 100;
}
```

B. The Zero-Blast Audio Hook (Content Script)

This logic secures the audio context and immediately mutes it before the user can hear anything.

```
function hookMediaElement(mediaEl) {
    if (mediaEl.dataset.isHooked === 'true') return;

    // Cross-origin fix for strict sites
    mediaEl.crossOrigin = "anonymous";

    const audioCtx = new (window.AudioContext || window.webkitAudioContext)();
    const source = audioCtx.createMediaElementSource(mediaEl);
    const gainNode = audioCtx.createGain();

    // ZERO-BLAST: Instantly set to 0 to prevent loud spikes
    gainNode.gain.value = 0;

    source.connect(gainNode);
    gainNode.connect(audioCtx.destination);

    // Save reference for future manipulation
    mediaEl.dataset.isHooked = 'true';
    window.extensionGainNode = gainNode;
}
```

5. Recommended Step-by-Step AI Prompts

Step	Prompt for Cursor
1. Project Setup	"Set up a Manifest V3 Chrome Extension. Create manifest.json with 'scripting', 'activeTab', and 'storage' permissions. Create a basic popup.html and popup.js."
2. Content Script	"Write a content_script.js that finds all video/audio tags. Implement a MutationObserver to catch new media tags. When found, route them through an AudioContext GainNode and set initial gain to 0."
3. UI & Math	"Update popup.html with a slider and a number input. Implement the logarithmic function in popup.js so the slider gives high precision between 0-20%. Sync the slider and text input so updating one updates the other."
4. Communication	"Write the message passing logic. When the user changes the volume in the popup, send a message to content_script.js to update the GainNode using 'exponentialRampToValueAtTime' to prevent audio popping."
5. Shortcuts & OSD	"Add chrome.commands to the manifest for Up/Down/Mute. In the background worker, listen for these and pass them to the content script. Have the content script render a temporary CSS-styled On-Screen Display (OSD) showing the new volume."

> **Important Skill Note: Exponential Ramping**
> When changing volume via digital gain, snapping directly from 0.1 to 0.5 causes a harsh "pop" or "click" sound due to wave discontinuity. Ensure Cursor uses `gainNode.gain.exponentialRampToValueAtTime(targetVolume, audioCtx.currentTime + 0.1)` for smooth transitions.